

Understanding how SQL Injection (SQLi) vulnerabilities occur in WordPress plugins is one of the best ways to level up your plugin development security.

Here is a breakdown of why CVE-2025-26943 happens, how to fundamentally prevent it, and what the code looks like before and after fixing it.

1. Why This Vulnerability Happens

In the case of CVE-2025-26943 within the *Easy Quotes* plugin (affecting versions up to 1.2.2), the flaw boils down to two core issues: **unsafe string concatenation** and a **lack of query preparation**.

When a plugin needs to look up data based on what a user types (like searching for a specific quote ID or author), developers sometimes dynamically build a SQL query like this:

```
PHP
// VULNERABLE APPROACH
$query "SELECT * FROM wp_easy_quotes WHERE quote_id = " $_GET 'id'
```

Because the input goes straight from the URL string into the database parser without being checked, the database can't tell the difference between the developer's instructions and the user's data.

If an attacker passes a malicious payload (like `1 OR 1=1` or time-based sleep commands) instead of a simple ID number, the database executes the attacker's commands instead. In this plugin, it allowed unauthenticated users to perform **Blind SQL Injection**, meaning they could systematically extract sensitive data (like administrator password hashes) based on how the server responded.

2. How Developers Should Fix It

WordPress provides a robust built-in database abstraction class called `$wpdb`. To secure raw SQL queries, developers should always implement two practices:

- **Use Parameterization via `$wpdb->prepare()`:** This acts like a blueprint. It forces the database engines to compile the query structural framework *before* inserting user data, treating the input strictly as a literal value, never as executable code.
- **Enforce Strict Input Sanitization/Typecasting:** If you expect an integer ID, explicitly force it to be an integer before it ever touches a database logic path.

3. Secure Coding Example

Let's look at how code transitions from vulnerable to fully secure using the native WordPress patterns.

The Vulnerable Code

In the unsafe version, the code blindly drops a \$_POST or \$_GET variable directly into the string query.

```
PHP
function get_quote_vulnerable()
{
    global $wpdb;
    $table_name = $wpdb->prefix . 'easy_quotes';

    // BAD: Directly grabbing input and sticking it into the query string
    $quote_id = $_POST['quote_id'];

    // Attackers can manipulate the layout of this query entirely
    $query = "SELECT * FROM $table_name WHERE id = '$quote_id'";
    return $wpdb->get_row($query);
}
```

The Secure Fixed Code

To fix it, we sanitize the type and wrap the execution inside the \$wpdb->prepare() method using type placeholders (%d for integers, %s for strings).

```
PHP
function get_quote_secure()
{
    global $wpdb;
    $table_name = $wpdb->prefix . 'easy_quotes';

    // 1. Sanitize/Cast input immediately based on expected type
    if (!isset($_POST['quote_id'])) {
        return null;
    }

    $quote_id = absint($_POST['quote_id']); // Forces the value to be a non-negative integer

    // 2. Use placeholders (%d for integer, %s for string) instead of direct concatenation
```

```
$query = $wpdb->prepare(
    "SELECT * FROM $table_name WHERE id = %d"
    $quote_id
)

// 3. Execute the safely compiled query
return $wpdb->get_row($query)
```

Why this works: Even if an attacker passes text like `4 UNION SELECT user_pass FROM wp_users`, `$wpdb->prepare` forces the database engine to look for an ID that literally equals that entire string, or the `absint()` function will reduce it to a safe integer, safely neutralizing the injection attempt.